Constitutional Computation

Computing Through Authorized Transformation

Every machine changes state. Constitutional machines must also justify it.

Author
Adam Ableman
Constitutional Computing
Contact: ableman.research@gmail.com

Abstract

Modern computation is founded upon models that describe how state changes occur. Turing Machines describe state transitions over symbolic tapes. Lambda Calculus describes computation through function application. Interaction systems describe computation through local graph rewriting. These models differ in representation and execution, yet share a common assumption:

> Computation concerns what can happen.

Legitimacy, authority, governance, and accountability are typically treated as external concerns, layered atop computational substrates after execution occurs.

This paper proposes **Constitutional Computation**, a computational paradigm in which legitimacy is elevated to a first-class computational primitive. Computation is modeled not merely as state transformation, but as **authorized transformation**. Effects are not directly executed. They are petitioned, evaluated against a constitution, admitted or denied, receipted, and only then allowed to alter state.

Constitutional Computation introduces formal structures for petitions, capabilities, constitutions, admission judgments, authority relations, effect semantics, time ordering, and receipts. The resulting model binds causality and legitimacy into a single computational process. State transitions become constitution-bounded rather than merely mechanically bounded.

The implications extend beyond governance and compliance. Constitutional Computation provides foundations for explainable software, governed artificial intelligence, capability-secure systems, semantic observability, autonomous agents, and future computational substrates in which execution and justification are inseparable.

---

1. Introduction

Software systems increasingly operate in domains where legitimacy matters as much as correctness.

Applications move money.

Agents invoke tools.

Cloud systems provision infrastructure.

Artificial intelligence systems perform actions in the world.

Yet the underlying computational models remain primarily concerned with causality.

They answer:

> What state follows from this state?

Modern systems increasingly require an additional question:

> Under what authority is this transformation admissible?

Traditional computation treats legitimacy as metadata.

Constitutional Computation treats legitimacy as part of the computation itself.

---

2. The Missing Axis of Computation

Every major computational model describes causality.

A Turing Machine describes:

```
State A
-> Transition
-> State B
```

Lambda Calculus describes:

```
Expression A
-> Reduction
-> Expression B
```

Interaction Systems describe:

```
Structure A
-> Rewrite
-> Structure B
```

Each determines what transformations are mechanically possible.

Constitutional Computation introduces a second axis:

`Can the transformation occur?`

`Is the transformation admissible?`

The first concerns mechanics.

The second concerns legitimacy.

The central claim of this paper is that legitimacy can be incorporated directly into the computational substrate.

---

3. Formal Model

A Constitutional Computation system is defined as:

$$\mathcal{C} = (S, P, Id, A, C, R, \Gamma, \vdash, \prec)$$

where:

- S is a state space
- P is a set of petitions
- Id is a set of actor identities
- A is a set of authorities
- C is a set of capabilities
- R is a set of receipts
- $\Gamma = (\Gamma_{adm}, \Gamma_{eff})$ is a constitution, decomposed into admission judgment and effect interpreter (see Section 3.1)
- $\vdash$ is the admission relation
- $\prec$ is an ordering relation over receipts and constitutional events

A constitutional transition is defined as:

$$(S_t, p) \overset{\Gamma}{\Rightarrow} (S_{t+1}, r, \pi)$$

iff:

$$\Gamma, S_t \vdash p \Downarrow (admit, \pi)$$

and

$$S_{t+1} = \Gamma_{eff}(S_t, p)$$

and

$$r = receipt(p, admit, \pi, H(r_{t-1}), ts, ver(\Gamma))$$

where:

- S_t is the current state at logical time t
- p is a petition
- π is the proof object constructed by Γ_{adm} during admission
- r is a receipt
- Γ_{eff} is the constitution-constrained effect interpreter

Execution is conditional upon proof of admissibility. The proof π is constructed during admission, not supplied externally. A separate verification mode permits externally supplied proofs and is treated in Section 3.1.

The constitution constrains not merely whether execution happens, but how admitted effects are interpreted.

3.1 Formal Refinement

The model above can be presented in a tighter form that eliminates redundant duality between the functional and relational presentations of Γ, makes proof construction explicit, structures the state space, integrates the effect interpreter into Γ, and collapses the entire model into a single transition rule.

Relational admission as primary. Treat the relational form as primary:

$$\Gamma, S_t \vdash p \Downarrow (decision, \pi)$$

Here π is constructed by Γ during admission rather than supplied externally. The functional notation $\Gamma(\cdot)$ remains valid as an implementation surface but is not a separate formal object. A variant supports externally supplied proofs for verification rather than construction:

$$\Gamma, S_t \vdash (p, \pi_{claim}) \Rightarrow decision$$

This cleanly separates proof construction from proof verification.

Constitution as admission-effect pair. The constitution is structured as:

$$\Gamma = (\Gamma_{adm}, \Gamma_{eff})$$

where Γ_{adm} is the admission judgment and Γ_{eff} is the effect interpreter:

$$\Gamma_{adm}, S \vdash p \Rightarrow S' = \Gamma_{eff}(S, p)$$

Structured state. State is decomposed into typed components:

$$S = (S_{data}, S_{auth}, S_{cap}, S_{meta})$$

where S_{data} holds application data, S_{auth} holds authority assignments, S_{cap} holds capability state including revocations, and S_{meta} holds constitution version and metadata. Admission and effect interpretation may read or write to any component, but the decomposition prevents hidden state channels: capability revocation lives in S_{cap}, constitution version lives in S_{meta}, and these are not silently smuggled through S_{data}.

Unified constitutional step. The entire model collapses into a single transition relation:

$$(S_t, p) \overset{\Gamma}{\Rightarrow} (S_{t+1}, r, \pi)$$

where r commits the decision and π explains it. The system is then defined as the closure of this relation over admitted petitions. This gives one semantic object, one transition rule, and maximal compression of the formal apparatus.

Definition (System). The Constitutional Computation system $\mathcal{C}$ is defined as the reflexive transitive closure of the constitutional step relation:

$$\mathcal{C} = closure(\overset{\Gamma}{\Rightarrow})$$

Minimality. The tuple (p, Γ, r, π) is minimal in the sense that removing any component eliminates either admissibility, interpretability, or verifiability of the transition: p carries the intent to be admitted, Γ carries the policy under which admission is decided, r carries the commitment of the decision, and π carries the justification that admits checking.

Reachability, composition, the minimal runtime, and the worked example may all be presented as instances or trajectories within this closure. The remainder of this paper continues to use the longer notations where they aid exposition, with the understanding that they are projections of this unified rule onto specific facets (admission, effect, ordering, receipt).

---

4. Constitutions

The constitution is the central governing object.

A constitution is defined as a state-indexed admission judgment that constructs a decision and a proof:

$$\Gamma, S_t \vdash p \Downarrow (decision, \pi)$$

where S_t is current state, p is a petition, and π is the proof object produced by admission (not supplied as input). The decomposition $\Gamma = (\Gamma_{adm}, \Gamma_{eff})$ from

Section 3.1 makes Γ_{adm} the carrier of this judgment and Γ_{eff} the carrier of effect interpretation.

The constitution may be:

- static
- stateful
- versioned
- self-amending

depending upon implementation.

For deterministic constitutional systems:

$$\Gamma, S_t \vdash p \Downarrow (decision, \pi)$$

must always yield the same *decision* and a deterministically constructed π for identical inputs.

More sophisticated constitutions may be expressed as proof systems. In that case, when state is implicit and admission is read as type derivation, the judgment may be abbreviated as:

$$\Gamma \vdash p : \tau$$

where (τ) represents a capability type or admissibility proof. This form is a notational abbreviation of the canonical state-indexed judgment, not a competing judgment.

Admission becomes proof construction rather than simple rule evaluation.

---

5. Constitution-Constrained Effects

A critical requirement of Constitutional Computation is that effect semantics are constrained by the constitution.

It is insufficient to define:

$$S' = \delta(S, p)$$

if (δ) is external to constitutional interpretation.

Otherwise, the same admitted petition could produce divergent states under different runtimes.

Constitutional Computation therefore requires:

$$\Gamma, S_t \vdash p \Downarrow (admit, \pi) \Rightarrow S' = \Gamma_{eff}(S_t, p)$$

This means admissibility and effect interpretation are coupled.

The constitution governs:

- whether an effect may occur
- how the effect is interpreted
- which state transition is legitimate
- which runtime semantics are valid

The machine does not merely ask:

> May this operation execute?

It also asks:

> What does this operation constitutionally mean?

---

6. Time and Ordering

Constitutional systems require an ordering model.

Receipts are ordered by:

$$r_i \prec r_j$$

meaning receipt (r_i) precedes receipt (r_j).

A logical timestamp may be defined as:

$$ts(r)$$

The constitution evaluates petitions against ordered state:

$$\Gamma, S_t \vdash p \Downarrow (decision, \pi)$$

Receipts may form a hash-linked lineage:

$$h_t = H(h_{t-1} \,\|\, r_t)$$

This provides replayability, tamper evidence, and deterministic reconstruction.

Without ordering, simultaneous petitions, conflicts, replay, and receipt verification remain underspecified.

---

7. Deterministic and Concurrent Modes

Constitutional systems may operate in deterministic or concurrent modes.

7.1 Deterministic Mode

Deterministic mode imposes a total order over petitions.

$$p_1 \prec p_2 \prec p_3$$

This produces replayable execution:

$$fold(R_{\leq t}^{commit}) = S_t \quad \text{iff} \quad \Gamma \text{ and } \prec \text{ are deterministic}$$

where $R_{\leq t}^{commit}$ is the subset of receipts recording admitted effects, and $fold$ is defined by iterative application of the constitutional effect interpreter:

$$fold(R_{\leq t}^{commit}) = \Gamma_{eff}(\ldots \Gamma_{eff}(\Gamma_{eff}(S_0, r_1.p), r_2.p) \ldots, r_t.p)$$

applied to the commit receipts in the order $\prec$.

Replayability requires both a deterministic effect interpreter Γ_{eff} and a stable ordering $\prec$. Without these, replay is not guaranteed.

Deterministic mode is useful for:

- financial ledgers
- audit-sensitive systems
- compliance workflows
- reproducible governance

7.2 Concurrent Mode

Concurrent mode permits partial ordering.

Two petitions may execute concurrently when they do not conflict:

$$(p_i \parallel p_j) \land \neg(p_i \# p_j) \Rightarrow commutative(p_i, p_j)$$

where:

$$\begin{aligned} & p_i \# p_j \\ & \iff \\ & \neq g\ commutative(\Gamma_{eff}(\cdot, p_i), \Gamma_{eff}(\cdot, p_j)) \end{aligned}$$

or equivalently at the state level:

$$\exists S : \ \Gamma_{eff}(\Gamma_{eff}(S, p_i), p_j) \neq \Gamma_{eff}(\Gamma_{eff}(S, p_j), p_i)$$

This grounds conflict in semantics rather than examples.

Concurrent mode requires conflict resolution policies inside (Γ).

---

8. Petitions

The petition is the fundamental computational object.

A petition is defined as:

$$p = (i, id, a, c, e, \sigma)$$

where:

- (i) = intent
- (id) = actor identity
- (a) = authority basis
- (c) = capability
- (e) = proposed effect
- (σ) = supporting evidence

Intent describes purpose.

Effect describes proposed state transformation.

Identity describes the actor.

Authority describes the source of legitimacy.

These must remain distinct.

A petition must satisfy:

$$wellFormed(p) \Rightarrow a \models c \wedge bind(id, a)$$

where:

$$bind \subseteq Id \times A$$

meaning the asserted authority basis must legitimately support the requested capability, and the asserted identity must be bound to that authority basis. Without identity-authority binding, an identity could assert arbitrary authority basis.

Unlike instructions, petitions are not executed directly.

They are evaluated.

---

9. Authority and Capabilities

Authority and capability are separate concepts.

Authority answers:

> Why may this actor act?

Capability answers:

> What may this actor do?

The authority relation is defined as:

$$\models \subseteq A \times C$$

$$a \models c$$

means authority basis (a) supports capability (c).

A capability is defined as:

$$c = (op, resource, constraints)$$

Examples:

```
(database.read,
 customer_records,
 region=EU)

(funds.transfer,
 treasury_account,
 amount < $10,000)
```

Capabilities support attenuation:

$$c' \subseteq c$$

where delegated capabilities are weaker than their parents.

Delegation is defined as:

$$A_1 \xrightarrow{delegate(c)} A_2$$

A delegated authority chain is valid only when attenuation is preserved:

$$a_0 \models c \land a_0 \to^* a_n \Rightarrow a_n \models c'$$

where:

$$c' \subseteq c$$

Constitutional systems may implement:

- revocable capabilities
- time-limited capabilities
- affine capabilities
- linear capabilities
- duplicable capabilities

depending on constitutional requirements.

---

10. Admission

Admission is the constitutional equivalent of evaluation.

A petition may be:

`admitted`

or:

`denied`

Admission is represented formally using the primary state-indexed judgment:

$$\Gamma, S_t \vdash p \Downarrow (\mathit{decision}, \pi)$$

Only admitted petitions may produce effects.

This creates a constitutional boundary between intention and transformation.

10.1 Soundness

Admission must be sound.

$$\Gamma, S_t \vdash p \Rightarrow p \text{ satisfies constitutional constraints under } S_t$$

Soundness ensures that admitted petitions are constitutionally valid.

10.2 Completeness

Some systems may additionally require completeness.

$$p \text{ valid under } S_t \Rightarrow \Gamma, S_t \vdash p$$

Completeness ensures that all valid petitions are admitted.

Many practical constitutional systems may choose soundness over completeness.

A conservative system may deny some valid petitions in order to prevent unauthorized transformation.

---

11. Proof Objects

A proof object is defined as:

$$\pi : Proof(p : admit)$$

or, in denial cases:

$$\pi : Proof(p : deny)$$

Proof objects must be checkable.

They may include:

- rule traces
- capability derivations
- delegation chains
- state predicates
- cryptographic attestations
- policy evaluation logs

Proofs may be minimal or expansive.

Minimal proofs reduce storage.

Expansive proofs improve auditability.

Advanced systems may use:

- succinct proofs
- zero-knowledge receipts
- proof compression
- verifiable computation

The proof object connects constitutional logic to computational verification.

Formally, proof construction implies the existence of a verification predicate:

$$\Gamma, S_t \vdash p \Downarrow (admit, \pi) \Rightarrow verify(\pi, p, \Gamma, S_t)$$

This closes the loop from construction to verification to soundness: admission produces π, π is independently checkable against (p, Γ, S_t), and soundness (Section 13.1) follows from the existence of a valid π.

---

12. Receipts

Receipts provide durable evidence that constitutional evaluation occurred.

A receipt is defined as:

$$r = (p, decision, \pi, h, ts, ver)$$

where:

- (p) = petition
- $(decision)$ = admission outcome
- (π) = proof trace
- (h) = receipt linkage hash
- (ts) = logical timestamp
- (ver) = constitution version under which Γ evaluated p

Receipts may form a cryptographically verifiable lineage graph.

Constitutional systems maintain the invariant:

$$(S_t, p) \overset{\Gamma}{\Rightarrow} (S_{t+1}, r, \pi) \Rightarrow r.decision = admit$$

No effect may exist without an admitting receipt.

Receipts must also correspond to the state they purport to commit:

$$r \rightsquigarrow S_{t+1} \Rightarrow S_{t+1} = \Gamma_{eff}(S_t, r.p)$$

This prevents forged admission receipts without execution, and prevents divergence between the receipt ledger and actual state.

Receipts must also be admission-consistent: an admit-receipt must correspond to an actual admission judgment:

$$r.decision = admit \Rightarrow \Gamma_{r.ver}, S_{r.ts} \vdash r.p$$

This prevents hash-valid receipts that record admission decisions never made by Γ.

Two receipt classes exist.

12.1 Authorization Receipt

Produced before execution.

Records admissibility.

12.2 Commit Receipt

Produced after execution.

Records successful state transition.

This distinction separates legitimacy from completion.

12.3 Denial Receipt

Denials are also receipted.

$$r.decision = deny$$

Denial receipts preserve evidence of attempted but refused transformations.

A denied petition may be revised:

$$p \xrightarrow{revise} p'$$

or escalated through a constitutionally defined appeals path.

---

13. Constitutional Reachability

Traditional computation defines reachable states mechanically.

Constitutional Computation defines reachable states constitutionally.

$$Next_\Gamma(S_t) = \{\Gamma_{eff}(S_t, p) \mid \Gamma, S_t \vdash p\}$$

This is equivalent to the existence of a constitutional step:

$$S_{t+1} \in Next_\Gamma(S_t) \iff \exists p, r, \pi : \ (S_t, p) \overset{\Gamma}{\Rightarrow} (S_{t+1}, r, \pi)$$

Reachability and transition semantics are thus a single object viewed from two angles.

Classical transition systems define reachability mechanically:

$$Next(S) = \{\delta(S, i)\}$$

Constitutional Computation acts as a restriction operator over transition systems.

It defines the subset of mechanically possible futures that are constitutionally admissible.

The future of the system is therefore bounded by legitimacy.

13.1 Named Properties

The model supports four named properties, each following directly from the invariants stated above and in Appendix B. These are presented as propositions rather than theorems because they are definitional consequences of the model rather than results requiring proof under a fixed constitutional logic.

Proposition (Authorization Safety). Every transition has an admitted petition:

$$(S_t, p) \overset{\Gamma}{\Rightarrow} (S_{t+1}, r, \pi) \Rightarrow \Gamma, S_t \vdash p$$

This follows from the transition rule itself: the constitutional step relation is defined only over petitions admitted by Γ_{adm}.

Proposition (Receipt Completeness). State is fully recoverable from the commit receipts in the ledger:

$$S_{t+1} = fold(R^{commit}_{\leq t})$$

where $R^{commit}_{\leq t} \subseteq R_{\leq t}$ is the subset of receipts recording admitted effects. Authorization receipts and denial receipts are preserved in $R_{\leq t}$ as evidence of constitutional evaluation but do not contribute to $fold$. This follows from the state-receipt consistency invariant and the determinism of Γ_{eff} under stable ordering $\prec$.

Proposition (Constitutional Soundness). Admission implies constitutional satisfaction:

$$\Gamma, S_t \vdash p \Rightarrow p \models \Gamma \text{ at } S_t$$

This follows from the definition of Γ_{adm} as a judgment over Γ.

Proposition (Replay Determinism). If Γ_{eff} and $\prec$ are deterministic, then for every t:

$$fold(R_{\leq t}^{commit}) = S_t$$

is unique. This follows from the determinism of Γ_{eff} under stable ordering: the iterative application defining $fold$ produces a single value at each step, and the receipt ledger thereby commits the system to a single reconstructible state trajectory. Replay is therefore a property of the model, not merely a feature of an implementation.

Each proposition is stated informally; formal proofs would require fixing a specific constitutional logic and are left to subsequent work.

13.2 Computational Class

Constitutional Computation defines a *restricted transition system*: the set of transitions admissible under Γ is a subset of the mechanically possible transitions of the underlying substrate.

$$TS_\Gamma \subseteq TS$$

Equivalently, the language $\mathcal{L}_\Gamma$ of constitutionally reachable trajectories is a restriction of the language $\mathcal{L}$ of mechanically reachable trajectories:

$$\mathcal{L}_\Gamma \subseteq \mathcal{L}$$

This positions Constitutional Computation alongside type systems (which restrict the language of programs), effect systems (which restrict the language of effects), and logical frameworks (which restrict the language of admissible derivations). In each case, a substrate's expressive freedom is constrained by a discipline that makes some property of the resulting computation legible.

---

14. Composition

Constitutional systems must support composed actions.

Sequential composition:

$$p_1 ; p_2$$

is valid only when both petitions transition consistently:

$$(S_t, p_1) \overset{\Gamma}{\Rightarrow} (S_{t+1}, r_1, \pi_1) \ \wedge \ (S_{t+1}, p_2) \overset{\Gamma}{\Rightarrow} (S_{t+2}, r_2, \pi_2)$$

In particular, $r_1.decision = admit$ is necessary but not sufficient: p_2 must also be admissible under the state S_{t+1} produced by p_1.

Parallel petitions require conflict analysis.

Define:

$$p_i \# p_j$$

when petitions are non-commutative.

Examples include:

- competing writes
- double-spend attempts
- conflicting authority assignments
- simultaneous constitutional amendments

Constitutional runtimes may support:

- atomic petition sets
- transactional petitions
- multi-petition admission
- two-phase admission and commit
- compensating petitions

as first-class constructs.

---

15. Constitutional Evolution

Constitutions may evolve.

But constitutional evolution must itself be governed.

An amendment petition is defined as:

$$p_{amend} \in P$$

If admitted:

$$\Gamma, S_t \vdash p_{amend} \Rightarrow \Gamma'$$

The amendment transition must produce a receipt and remain inside the same transition system:

$$
\begin{aligned}
&(S_t, p_{amend}) \overset{\Gamma}{\Rightarrow} (S_{t+1}, r, \pi) \\
&\wedge\ \Gamma \\
&\to \Gamma'
\end{aligned}
$$

where $S_{t+1}.S_{meta}.ver = v + 1$ and $S_{t+1}.S_{data} = S_t.S_{data}$. Amendment does not change application data but does increment the constitution version in S_{meta}, which subsequent admission judgments will read via $currentVersion$.

The constitution version is operationally bound into the transition rule. Each receipt r records the version $v = r.ver$ under which it was evaluated, and subsequent petitions evaluate under the constitution version current at their timestamp:

$$(S_t, p) \overset{\Gamma_v}{\Rightarrow} (S_{t+1}, r, \pi) \quad \text{where} \quad v = currentVersion(t)$$

Equivalently, for any receipt r in the ledger, the admission that produced it was evaluated under $\Gamma_{r.ver}$ at state $S_{r.ts}$:

$$\Gamma_{r.ver}, S_{r.ts} \vdash r.p$$

This makes versioning operationally binding rather than only documentary.

This prevents governance root-of-trust ambiguity.

A constitutional system must define:

- who may amend (Γ)
- which authority basis supports amendment
- whether amendments are immediate or delayed
- whether prior receipts remain evaluated under historical constitutions
- how constitutional versions are addressed

A receipt should therefore include the constitution version used for evaluation. Formally:

$$ver : R \to V$$

and:

$$\Gamma_v \vdash p \Rightarrow r.ver = v$$

Replay across amendments must apply the constitution version recorded in each receipt:

$$fold(R) \text{ applies } \Gamma_{r.ver} \text{ for each } r \in R$$

Without versioned replay, execution under amended constitutions is undefined.

---

16. Worked Example: Competing Treasury Transfers

Consider two agents.

Agent A possesses delegated authority to transfer up to $5,000.

Agent B possesses delegated authority to transfer up to $50,000.

Current balance:

```
$7,000
```

Two petitions arrive.

Petition (P_1):

```
Intent:
    vendor_payment

Identity:
    agent_A

Authority:
    delegated_accounts_payable_role

Capability:
    funds.transfer(amount <= 5000)

Effect:
    transfer $5,000 to Vendor X
```

Petition (P_2):

```
Intent:
    vendor_payment

Identity:
    agent_B

Authority:
    delegated_treasury_role

Capability:
    funds.transfer(amount <= 50000)

Effect:
    transfer $6,000 to Vendor Y
```

The petitions conflict:

$$P_1 \# P_2$$

because both consume from the same balance.

Under deterministic ordering:

$$P_1 \prec P_2$$

the constitution evaluates:

$$\Gamma, S_0 \vdash P_1 \Downarrow (admit, \pi_1)$$

because:

- identity is authenticated
- authority is valid
- capability is valid
- delegation is attenuated
- sufficient balance exists

Authorization receipt (R_1^a) is generated.

The effect executes:

$$S_1 = \Gamma_{eff}(S_0, P_1)$$

The balance becomes:

`$2,000`

Commit receipt (R_1^c) is generated.

The constitution then evaluates (P_2):

$$\Gamma, S_1 \vdash P_2 \Downarrow (deny, \pi_2)$$

because the balance constraint is no longer satisfied.

Denial receipt (R_2^d) records:

- valid identity
- valid authority
- valid capability
- failed state predicate
- ordering dependency

The resulting receipt graph explains:

- why one transfer occurred
- why the second failed
- which authority participated
- which capability was exercised
- which constitutional rule applied
- which state predicate prevented the second transfer

The explanation emerges from computation itself.

---

17. Constitutional Observability

Traditional observability answers:

> What happened?

Constitutional observability answers:

> Why did it happen?

Because every effect originates from an admitted petition, receipt graphs become semantic execution traces.

Let:

$$Q(R) \to explanations$$

be a query over receipt graph (R).

A provenance query is defined over the effect $e = p.e$ of a petition p:

First define the receipt-of relation: for a petition p that produced receipt r_p,

$$r \rightsquigarrow p \;:=\; r \prec^* r_p$$

Then the provenance of a petition is:

$$prov(p) = \{r \mid r.p = p \vee r \rightsquigarrow p\}$$

An authority trace may be defined as:

$$auth(e) = chain(a)$$

Causal closure over a receipt is defined as the transitive closure of $\prec$:

$$closure(r) = \{r' \mid r' \prec^* r\}$$

Causal closure is essential for full provenance and audit completeness.

A denial query may be defined as:

$$deny(e) = \{r \mid r.decision = deny \wedge r.p.e = e\}$$

Observability becomes structured inquiry over constitutional history.

Execution, explanation, and governance share the same substrate.

---

18. Artificial Intelligence

Autonomous agents create legitimacy challenges.

An agent may possess:

- memory
- tools
- databases
- network access
- delegated authority

Constitutional Computation converts agent actions into petitions.

Tool invocations become capability exercises.

Memory modifications become authorized transformations.

Delegation becomes explicit and receipted.

This creates a framework for governed agency in which explanation is intrinsic.

An agent does not simply call a tool.

It petitions to exercise a capability.

---

19. Safety and Liveness

Constitutional systems prioritize safety.

Safety condition:

> No unauthorized transformation occurs.

Formally:

$$(S_t, p) \overset{\Gamma}{\Rightarrow} (S_{t+1}, r, \pi) \Rightarrow \Gamma, S_t \vdash p$$

However, safety alone is insufficient.

A perfectly governed system that never executes is useless.

Liveness in its strongest form would require:

$$\forall p, S_t,\ (\Gamma, S_t \vdash p) \Rightarrow \Diamond\ \exists S_{t+1} :\ (S_t, p) \overset{\Gamma}{\Rightarrow} (S_{t+1}, r, \pi)$$

meaning every admitted petition eventually produces a transition. This is unrealistic under contention. A more realistic relaxation is:

$$\forall p, S_t,\ (\Gamma, S_t \vdash p) \Rightarrow \Diamond\ (\exists S_{t+1} :\ (S_t, p) \overset{\Gamma}{\Rightarrow} (S_{t+1}, r, \pi)\ \vee\ revoke(p))$$

meaning every admitted petition eventually executes or is explicitly revoked. Constitutional systems may further scope liveness per authority or per capability class.

Constitutional systems must therefore manage:

- governance deadlock
- admission bottlenecks
- constitutional contention
- proof evaluation delays
- conflict resolution starvation

The practical system must balance legitimacy and throughput.

---

20. Cost Model

Constitutional Computation introduces overhead.

Execution cost becomes:

$$T_{constitutional} = T_{compute} + T_{admission} + T_{receipt}$$

The admission term itself decomposes as:

$$T_{admission} = O(|\pi| + eval(\Gamma))$$

In the worst case, policy evaluation over an expressive constitutional language may be NP-hard. Practical systems must therefore use restricted constitutional languages to keep $eval(\Gamma)$ tractable.

Additional costs include:

- proof evaluation
- receipt storage
- capability validation

- lineage maintenance
- conflict analysis
- constitutional versioning

Receipt graph growth is at least linear in the number of petitions:

$$|R| = O(|P|)$$

Proof traces may create superlinear storage pressure if they include large evidence structures.

Mitigation strategies include:

20.1 Compression

$$R' = compress(R)$$

preserving constitutional invariants.

20.2 Checkpointing

$$S_k = fold(R^{commit}_{\leq k})$$

allowing replay from checkpoints rather than genesis.

20.3 Amortization

Systems may use:

- batched petitions
- cached proofs
- capability pre-certification
- receipt compression
- admission memoization
- periodic state commitments

The objective is not free governance.

The objective is governance with bounded cost.

---

21. Failure Modes and Invariants

Constitutional systems are not immune to failure.

But each failure mode can be tied to an invariant.

21.1 Forged Authority

If authority does not support capability:

$$\neg(a \models c) \Rightarrow \Gamma \nvdash p$$

21.2 Capability Leakage

Delegated capabilities must preserve attenuation:

$$c_{delegated} \subseteq c_{source}$$

21.3 Receipt Tampering

If receipt hashes fail:

$$H(r_i) \neq h_{i+1} \Rightarrow \textit{invalid chain}$$

21.4 Governance Capture

A malicious constitution may admit harmful petitions.

Mitigation requires constitutional amendment controls, versioned receipts, root-of-trust protection, and meta-governance.

21.5 Delegation Explosion

Unbounded delegation can create uncontrolled authority propagation.

Mitigation requires:

- attenuation
- revocation
- expiry
- delegation depth limits
- receipt-backed delegation graphs

21.6 Constitutional Deadlock

Legitimate actions may be blocked indefinitely.

Mitigation requires liveness guarantees, escalation paths, and deadlock detection.

21.7 State-Receipt Divergence

If state mutates without a corresponding admitting receipt, or if a receipt records an effect that did not occur, the system loses constitutional integrity.

The system-wide integrity invariant is:

$$S_{t+1} = fold(R_{\leq t}^{commit})$$

where $R_{\leq t}^{commit}$ is the subset of receipts recording admitted effects, as in Section 13.1.

Any divergence between the receipt ledger and the actual state must be detected and treated as a constitutional fault.

---

22. Relationship to Existing Work

Constitutional Computation integrates ideas from multiple traditions while drawing a sharp distinction.

Capability systems constrain who can act.

Proof-Carrying Code proves program safety.

Blockchain systems order what happened.

Event sourcing records what happened.

Constitutional Computation determines what is allowed to happen before it happens.

22.1 Capability Systems

Constitutional Computation extends capability security by making capability exercise receipted, state-aware, and constitutionally evaluated.

22.2 Proof-Carrying Code

Constitutional Computation extends proof verification from program safety to operational legitimacy.

22.3 Event Sourcing

Constitutional Computation extends event logs into constitutional evidence structures where events must originate in admitted petitions.

22.4 Blockchain Systems

Constitutional Computation extends ledger concepts beyond ordering toward legitimacy, admissibility, and authority-bearing transformation.

22.5 Distributed Systems

Constitutional Computation extends causal ordering with authority ordering, intent ordering, and governance-aware conflict resolution.

---

23. Coherence as a Computational Objective

Modern systems maintain multiple conflicting models:

- permissions
- workflows
- documentation
- audit logs
- organizational charts
- runtime execution

These drift.

Constitutional systems seek coherence.

The mechanism is explicit:

1. Governance artifacts compile into (Γ).
2. Effect semantics are constrained by Γ_{eff}.
3. Execution occurs only through admitted petitions.
4. Every admission emits receipts.
5. Every effect has an admitting receipt.
6. Observability queries receipt graphs.
7. Constitutional amendments are themselves receipted.

Governance, execution, and explanation remain synchronized because they are generated by the same computational process.

Coherence becomes an emergent property of constitutional execution.

---

24. Minimal Constitutional Runtime

A minimal implementation separates evaluation, recording, and execution.

```
## evaluation depends on current state
decision, proof = evaluate(p, Gamma, S_t)

## authorization receipt captures the admission decision
auth_receipt = record_authorization(
    petition=p,
    decision=decision,
    proof=proof,
```

```
    timestamp=ts,
    constitution=Gamma.version
)
append(auth_receipt)

## only admitted petitions produce effects
if decision == "admit":
    S_next = Gamma_eff(S_t, p)

## commit receipt captures successful state transition
    commit_receipt = record_commit(
        petition=p,
        prior_state_hash=H(S_t),
        new_state_hash=H(S_next),
        timestamp=ts,
        constitution=Gamma.version
    )
    append(commit_receipt)
```

The runtime enforces the invariant:

$$receipt.ts = t \Rightarrow S_t \text{ is the evaluation state}$$

Without this invariant, concurrent admission can introduce ambiguity over which state was evaluated.

This minimal runtime contains the constitutional cycle:

```
petition
-> evaluation
-> proof
-> receipt
-> conditional effect
-> ordered history
```

From these primitives emerge:

- governance
- observability
- capability security
- receipt graphs
- constitutional execution

The constitutional cycle is simple despite its consequences.

---

25. Toward Constitutional Machines

Traditional computational systems evolve through mechanically permitted transformations.

Constitutional systems evolve through admissible transformations.

The primitive computational object is no longer the instruction.

It is the constitutional step itself:

$$(S_t, p) \overset{\Gamma}{\Rightarrow} (S_{t+1}, r, \pi)$$

This relation carries state, petition, constitution, successor state, receipt, and proof together as a single semantic object. The earlier presentations (p, Γ, r), (p, Γ, r, π), and the labeled transition form are projections of this primitive onto specific facets.

The resulting machine computes not merely state.

It computes state together with the legitimacy of its own evolution.

Conclusion

Constitutional Computation introduces legitimacy as a first-class computational primitive.

Traditional computational models define what transformations are mechanically possible.

Constitutional Computation defines what transformations are constitutionally reachable.

The central claim may be expressed formally:

$$Next_\Gamma(S_t) = \{\Gamma_{eff}(S_t, p) \mid \Gamma, S_t \vdash p\}$$

The future of a constitutional system is not bounded solely by mechanics.

It is bounded by legitimacy.

By elevating constitutions, petitions, admission judgments, capabilities, authorities, ordering, effect semantics, and receipts into the computational substrate itself, software systems gain intrinsic explainability, accountability, observability, and governance.

Computation therefore becomes the evolution of state through authorized transformation.

Stated in the language of formal systems: Constitutional Computation lifts transition systems into *justified transition systems*, where transitions are paired with admissibility proofs and constrained interpreters.

Equivalently, a Constitutional Machine is a labeled transition system:

$$(S_t) \xrightarrow{p\ /\ (r,\pi)} (S_{t+1})$$

with petition p as the label and the pair (r, π) as the annotation. The unified constitutional step and the LTS form are equivalent:

$$(S_t, p) \overset{\Gamma}{\Rightarrow} (S_{t+1}, r, \pi) \ \equiv \ (S_t) \xrightarrow{p\ /\ (r,\pi)} (S_{t+1})$$

This closes the loop between the operational-semantics presentation of Section 3.1, the custom transition notation used throughout the paper, and standard LTS framing. This places Constitutional Computation directly within the vocabulary of operational semantics, labeled transition systems, and process calculi.

In particular, Constitutional Computation is a conservative extension of labeled transition systems in which transitions are constrained to those admitting a justification under Γ. When Γ is trivially permissive, the constitutional transition system reduces to its underlying transition system; the constitutional structure imposes no restriction in the limit.

Appendix A: Primitive Objects

- **Object:** Constitution
  Definition: (Γ)
- **Object:** Petition
  Definition: $(p = (i, id, a, c, e, \sigma))$
- **Object:** Capability
  Definition: $(c = (op, resource, constraints))$
- **Object:** Authority relation
  Definition: $(\models \subseteq A \times C)$
- **Object:** Receipt
  Definition: $(r = (p, decision, \pi, h, ts, ver))$
- **Object:** Admission
  Definition: $(\Gamma, S_t \vdash p)$
- **Object:** Effect semantics
  Definition: $\Gamma_{eff}(S, p)$

- **Object:** Ordering
  Definition: $(r_i \prec r_j)$

Appendix B: Constitutional Invariants

Authorization Invariant:

$$(S_t, p) \overset{\Gamma}{\Rightarrow} (S_{t+1}, r, \pi) \Rightarrow r.decision = admit$$

Reachability Invariant:

$$S_{t+1} \in Next_\Gamma(S_t) \Rightarrow \exists p : \Gamma, S_t \vdash p$$

Capability Invariant:

$$wellFormed(p) \Rightarrow a \models c$$

Attenuation Invariant:

$$c_{delegated} \subseteq c_{source}$$

Receipt Integrity Invariant:

$$H(r_i) = h_{i+1}$$

Constitutional Evolution Invariant:

$$\Gamma \to \Gamma' \Rightarrow \exists r$$

Master Invariant. The state-receipt, admission-consistency, and replay-correctness invariants can be unified as a single closure property:

$$S_t = fold(R^{commit}_{\leq t}) \ \wedge \ \forall r \in R : \ (r.decision = admit \Rightarrow \Gamma_{r.ver}, S_{r.ts} \vdash r.p)$$

The first conjunct ensures the receipt ledger reconstructs current state; the second ensures every admitted receipt corresponds to a real admission judgment under the state at the time of issuance.

Corpus-Wide Reference Positioning

This paper argues that modern computation is founded upon models that describe how state changes occur. Its nearest external comparison fields are formal, governance, semantic. The sources in [E1], [E2], [E3], [E4], [E5], [E6], [E7] are cited as predecessor literature, adjacent standards, or counterpressure - not as evidence that they caused the Ableman framework or that they validate its novel claims. The comparison isolates where established work already supplies provenance, formal assurance, institutional constraint, empirical method, or engineering practice, and where this paper adds a distinct claim about admissibility, authority, semantic custody, or governed realization.

Within the Ableman corpus, the highest-priority current cross-links are [A1], [A2], [A3], [A4], [A5], [A6], [A7]. These are links among separately custodied works, not silent identity mergers. Their role is to expose mutual dependencies, terminology collisions, and places where a reader should traverse the corpus before treating this paper as self-sufficient.

Reference-pass counterevidence and limit search

The pass applied the following pressure tests: Formal assurance remains relative to the encoded model, trusted base, and refinement chain; proof does not remove specification error or unmodeled environmental behavior. A written constitution can legitimate domination as easily as constrain it unless amendment, participation, enforcement, and appeal remain effective in practice. Formal vocabularies and graph links make commitments explicit but cannot by themselves establish correct identity, shared interpretation, or lossless translation between contexts. These limits are not generic disclaimers: they identify the assumptions that a later empirical, formal, or implementation review must try to break.

Reference-pass influence and novelty boundary

No newly added external source is asserted to be a documented authorial influence unless the predecessor source already said so. `Predecessor` means earlier relevant work;

`adjacent` means a useful comparison; `standard` means an interoperability or assurance baseline; and `counterevidence` marks work that narrows or pressures a claim. Novelty therefore remains a reviewable, claim-level question rather than an inference from a low similarity score or an absent citation.

References

External scholarship, standards, and prior art

[E1] George C. Necula. "Proof-Carrying Code." *POPL '97*, 1997, pp. 106-119. DOI: 10.1145/263699.263712. https://doi.org/10.1145/263699.263712 **Relationship:** Predecessor for binding machine-checkable evidence to executable

artifacts; the present paper may demand broader semantic or constitutional obligations.

[**E2**] Xavier Leroy. "Formal Verification of a Realistic Compiler." *Communications of the ACM* 52(7), 2009, pp. 107-115. DOI: 10.1145/1538788.1538814. https://doi.org/10.1145/1538788.1538814 **Relationship:** Predecessor verified-compilation result used to distinguish proven translation preservation from broader governance claims.

[**E3**] Patrick Cousot and Radhia Cousot. "Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints." *POPL '77*, 1977. DOI: 10.1145/512950.512973. https://doi.org/10.1145/512950.512973 **Relationship:** Predecessor formal framework for sound approximation and fixpoint reasoning.

[**E4**] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers.* Addison-Wesley, 2002. https://lamport.azurewebsites.net/tla/book.html **Relationship:** Predecessor specification discipline for state machines, safety, liveness, and refinement.

[**E5**] Elinor Ostrom. *Governing the Commons: The Evolution of Institutions for Collective Action.* Cambridge University Press, 1990. DOI: 10.1017/CBO9780511807763. https://doi.org/10.1017/CBO9780511807763 **Relationship:** Predecessor institutional account of rule-governed collective action and polycentric governance.

[**E6**] Wil Waluchow and Dimitrios Kyritsis. "Constitutionalism." *Stanford Encyclopedia of Philosophy*, substantive revision 2023. https://plato.stanford.edu/entries/constitutionalism/ **Relationship:** Adjacent legal-philosophical account of limited authority; included to expose differences between political and computational constitutionalism.

[**E7**] Lawrence Lessig. *Code: Version 2.0.* Basic Books, 2006. https://codev2.cc/ **Relationship:** Predecessor argument that software architecture regulates behavior; the paper's contribution may be to make such regulation inspectable and revisable.

Ableman corpus

[**A1**] Adam Ableman. *MCIR: A Canonical Intermediate Representation for Governed Transformation.* Ableman Corpus current edition; work identity `PAPER.PUB.21095303`;
source SHA-256
`EA129ECBA95F7B07CAA7B33A54F91F52913C94A14DA227EB9A06CFD083E4E5BF`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A2**] Adam Ableman. *Governed Semantic Compilation.* Ableman Corpus current edition; work identity `PAPER.PUB.18453681`;

source SHA-256
5FF6C636550B04DB0E2F6F94C379BB75AA0BD8781E19B047D7D3BB6D49071AC2.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A3**] Adam Ableman. *Hybrid Semantic Compiler Architectures.* Ableman Corpus current edition; work identity PAPER.PUB.21228038;
source SHA-256
EBCD6DDDA9B547639AEBED79FB68069473CF1248E41C527B48C9624BBE2C0496.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A4**] Adam Ableman. *Admissibility Engineering.* Ableman Corpus current edition; work identity
AMCS-LOCAL-WORK-F1E33B5E973B0900;
source SHA-256
CDC80271268802CA5CB4DE3624B992FB2FF07D677BF945C4B349BD307C4C2A6E.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A5**] Adam Ableman. *Everything Is a Compiler.* Ableman Corpus current edition; work identity
AMCS-LOCAL-WORK-30FB8F95BD990A47;
source SHA-256
FE0F88A1F125317285BE4378E5F27ACC9FEB83ADE3FF329A3DA9F9CB026DF9A9.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A6**] Adam Ableman. *Ableman Constitutional Governance.* Ableman Corpus current edition; work identity
AMCS-LOCAL-WORK-79353AF163A4B0F1;
source SHA-256
FB0D9B2D3D54168FFE70B24308D1B2766D6D663ED24502D475E975939BE97C8B.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A7**] Adam Ableman. *Constitutional Theory.* Ableman Corpus current edition; work identity
AMCS-LOCAL-WORK-D2D97C4042FD281F;
source SHA-256
FACC256EDDE5A7D7EA18133CFC9C060CFB8C3811B0221E1969E62347F2BB69CF.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

Editorial source notes retained from predecessor

Turing, A. M. (1936). *On Computable Numbers, with an Application to the Entscheidungsproblem.*

Church, A. (1936). *An Unsolvable Problem of Elementary Number Theory.*

Lafont, Y. (1997). *Interaction Combinators.*

Lamport, L. (1978). *Time, Clocks, and the Ordering of Events in a Distributed System.*

Saltzer, J. H., & Schroeder, M. D. (1975). *The Protection of Information in Computer Systems.*

Denning, D. E. (1976). *A Lattice Model of Secure Information Flow.*

Miller, M. S. (2006). *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control.*

Necula, G. C. (1997). *Proof-Carrying Code.*

Ableman, A. (2026). *Universal Governance Kernel (UGK): Constitutional Governance for Software Systems.*

Ableman, A. (2026). *Closing the Gap: ACTIVE, CRYSTALLIZED, and ABSENT Governance in Modern Software.*

Corpus source and companion formats: https://github.com/AblemanMosaic/ableman-corpus
License: CC-BY-ND-4.0

DOI: 10.5281/zenodo.20624488